

SOLID PRINCIPI

Uvod

Ovaj dokument daje pregled usklađenosti priloženog UML dijagrama klasa sa SOLID principima. Analiza je bazirana na modelu koji sadrži klase **Korisnik**, **Putnik**, **Destinacija**, **Paket**, **Rezervacija**, **Plaćanje**, **Notifikacija**, **PlanPutovanja**, kao i enumeracije **Uloga**, **StatusPaketa**, **StatusRezervacije**, **MetodaPlaćanja**, **TipNotifikacije**. U modelu su dodatno definisani interfejsi (paketi/servisi) **AutentifikacijaPaket**, **PaketiServis**, **RezervacijaPaket**, **PlaćanjePaket**, **NotifikacijaPaket** i **PlanPutovanjaPaket**.

1. S - Single Responsibility Principle (Principi dobrog dizajna)

KLASA BI TREBALA IMATI SAMO JEDAN RAZLOG ZA PROMJENU.

Primjena u modelu:

U dijagramu su entitetske klase (npr. **Korisnik**, **Paket**, **Rezervacija**, **Plaćanje**, **Notifikacija**, **PlanPutovanja**) fokusirane na podatke i osnovna pravila. Složeniji procesi (kreiranje i otkazivanje rezervacije, obrada plaćanja, slanje potvrda) izdvojeni su u posebne **interfejse/servise** (npr. **RezervacijaPaket**, **PlaćanjePaket**, **NotifikacijaPaket**). Tako se promjene u načinu rada sistema rade na jednom mjestu, bez mijenjanja svih klasa.

2. O - Open/Closed Principle (Otvoreno zatvoren princip)

ENTITETI SOFTVERA (KLASE, MODULI, METODE) TREBALI BI BITI OTVORENI ZA NADOGRADNJU, ALI ZATVORENI ZA MODIFIKACIJE.

Primjena u modelu:

Sistem treba biti **otvoren za proširenja**, ali da se postojeći kod **ne mora mijenjati** svaki put kad dodamo novu opciju. U našem modelu to postizemo tako što koristimo **interfejse**. Na primjer, dodavanje nove metode plaćanja ili novog načina slanja notifikacija može se riješiti proširenjem implementacije servisa, dok klase poput **Rezervacija** i **Plaćanje** ostaju nepromijenjene. Enumeracije (npr. **MetodaPlaćanja**, **StatusRezervacije**) dodatno pomažu da se stanja i vrste jasno definišu i koriste kroz sistem.

3. L - Liskov Substitution Principle (Liskov princip zamjene)

PODTIPOVI MORAJU BITI ZAMJENJIVI NJIHOVIM OSNOVNIM TIPOVIMA.

Primjena u modelu:

Ako imamo više implementacija jednog interfejsa, svaka od njih mora se moći koristiti bez rušenja sistema. Iako dijagram ne koristi kompleksno nasljeđivanje među entitetima, LSP se primjenjuje kroz interfejse. Svaka implementacija interfejsa kao što su **RezervacijaPaket**, **PlaćanjePaket** ili **NotifikacijaPaket** može se zamijeniti drugom implementacijom, a da se pri tome ne naruši ispravnost rada sistema. Sistem se oslanja na ugovore koje interfejs propisuje, a ne na detalje konkretne realizacije

4. I - Interface Segregation Principle (Princip segregacije interfejsa)

KLIJENTI NE TREBA DA OVISE O METODAMA KOJE NEĆE UPOTREBLJAVATI.

Primjena u modelu:

Bolje je imati više **malih i jasnih interfejsa** nego jedan veliki koji sadrži sve. Umjesto jednog velikog interfejsa, model koristi više manjih, namjenskih interfejsa. Na primjer, **RezervacijaPaket** pokriva samo operacije vezane za rezervacije, **PlaćanjePaket** samo plaćanja, **NotifikacijaPaket** samo slanje potvrda, a **PlanPutovanjaPaket** samo generisanje plana putovanja. Tako klase i dijelovi sistema koriste samo ono što im stvarno treba i ne moraju zavisiti od metoda koje nikad ne koriste.

5. D - Dependency Inversion Principle (Princip inverzije ovisnosti)

MODULI VISOKOG NIVOA NE BI TREBALI OVISITI OD MODULA NISKOG NIVOA. OBA BI TREBALA DA OVISE OD APSTRAKCIJA.

MODULI NE BI TREBALI OVISITI OD DETALJA. DETALJI BI TREBALI BITI OVISNI OD APSTRAKCIJA.

Primjena u modelu:

Glavna poslovna logika ne treba direktno zavisiti od konkretnih detalja (npr. tačan način slanja e-maila ili tačan način plaćanja). Umjesto toga, treba zavisiti od **apstrakcija** (interfejsa). U našem dijagramu servisi su predstavljeni kao interfejsi, pa se ostatak sistema oslanja na njih, a ne na konkretne implementacije. To olakšava promjene i testiranje: možemo promijeniti način slanja notifikacija ili obrade plaćanja bez mijenjanja osnovnih klasa kao što su **Rezervacija** ili **Plaćanje**.

Zaključak

Na osnovu priloženog UML dijagrama klasa, model je dobro usklađen sa SOLID principima jer su **entiteti odvojeni od poslovnih procesa**, a ključne funkcionalnosti su smještene u **posebne servise (interfejse)**. Time se postiže da su klase jednostavnije, jasnije i lakše za održavanje. Sistem je spreman za proširenja (npr. nove metode plaćanja ili novi načini slanja notifikacija) bez velikih izmjena postojećih klasa. Također, korištenje manjih i specifičnih interfejsa olakšava testiranje i smanjuje zavisnosti među komponentama.

Ipak, postoji **rizik kršenja OCP-a** ako se nove metode plaćanja ili notifikacija budu dodavale kroz stalno proširivanje switch/case logike umjesto kroz nove implementacije. Također, **DIP** će

biti potpuno ispoštovan samo ako se u implementaciji zaista koristi zavisnost od apstrakcija (interfejsa), a ne od konkretnih implementacija (npr. direktan poziv konkretnog email ili payment API-ja).